Experiment No: 1

Experiment Name

Implementation of Text Tokenization and Stop Word Removal Using Python and NLTK

Objective

To split raw text into sentences and words using NLTK tokenizers, and to remove stop words so only content-bearing tokens remain.

Introduction

Tokenization is the first step of almost every NLP pipeline. It breaks a character stream into units (sentences, words, punctuation) that later stages can process. Stop words such as *the*, *is*, *to* occur very frequently but carry little meaning, so removing them shrinks the vocabulary and helps tasks like search, classification and keyword extraction. NLTK offers `sent_tokenize`, `word_tokenize` (Punkt/Treebank based) and a curated stop-word list for English.

Algorithm

1. Input the raw text.
2. Split into sentences with `sent_tokenize`.
3. Split into word tokens with `word_tokenize`.
4. Load the English stop-word set.
5. Keep each token only if its lower-case form is not a stop word and it is alphanumeric (this also drops punctuation).
6. Print the filtered list and token counts.

Code

```
import nltk
from nltk.tokenize import word_tokenize, sent_tokenize
from nltk.corpus import stopwords

text = ("Natural Language Processing enables computers to understand human language. "
        "It is widely used in chatbots, search engines, and translation systems!")
print("Original:", text)
print("\nSentence tokens:")
for s in sent_tokenize(text): print("  ", s)
words = word_tokenize(text)
print("\nWord tokens:", words)
stop = set(stopwords.words("english"))
filtered = [w for w in words if w.lower() not in stop and w.isalnum()]
print("\nAfter stop-word removal:", filtered)
print(f"\nTokens before: {len(words)}  after: {len(filtered)}")
```

Result and Discussion

Output:

```
Original: Natural Language Processing enables computers to understand human language. It is
widely used in chatbots, search engines, and translation systems!

Sentence tokens:
   Natural Language Processing enables computers to understand human language.
   It is widely used in chatbots, search engines, and translation systems!
```

```
Word tokens: ['Natural', 'Language', 'Processing', 'enables', 'computers', 'to', 'understand',
'human', 'language', '.', 'It', 'is', 'widely', 'used', 'in', 'chatbots', ',', 'search',
'engines', ',', 'and', 'translation', 'systems', '!']

After stop-word removal: ['Natural', 'Language', 'Processing', 'enables', 'computers',
'understand', 'human', 'language', 'widely', 'used', 'chatbots', 'search', 'engines',
'translation', 'systems']

Tokens before: 24  after: 15
```

Discussion: The text was split into 2 sentences and 24 word tokens (punctuation counted as tokens). After stop-word and punctuation removal 15 tokens remained, a reduction of 37.5%. Words such as *to, is, in, and, it* and the tokens `.`, `,`, `!` were removed, leaving the meaningful words (*Natural, Language, Processing, computers, chatbots, ...*). Note that case was kept for output, but the stop-word test used lower case so that *It* was still removed. Removing stop words is not always desirable, for example in sentiment analysis "not" is a stop word that flips meaning.

Conclusion

Tokenization and stop-word removal successfully cleaned the text, cutting the token count from 24 to 15 and leaving the meaningful words.

Experiment No: 2

Experiment Name

Implementation of Word Analysis and Word Generation in Natural Language Processing

Objective

To analyse the statistical properties of words in a text (frequency, length, lexical diversity) and to generate new word forms using affix rules and WordNet derivations.

Introduction

Word analysis studies how words are distributed in a corpus, which is the basis of frequency-based models such as TF-IDF. Word generation is the reverse process: producing valid word forms from a root by adding prefixes and suffixes (derivational morphology) or by looking up derivationally related forms in a lexical database like WordNet.

Algorithm

1. Tokenize the text, lower-case it and keep alphabetic tokens.
2. Compute total words, unique words, lexical diversity (unique/total), average word length, longest word.
3. Count word frequencies with `Counter` and list the most common.
4. Define lists of prefixes and suffixes.
5. For each root, concatenate every prefix/suffix with it to generate candidates.
6. For a verb, collect `derivationally_related_forms()` of all WordNet lemmas of all its synsets.

Code

```
# Word analysis (frequency, length, vocabulary) and word generation (affixes + WordNet
derivations)
from collections import Counter
from nltk.tokenize import word_tokenize
from nltk.corpus import wordnet as wn

text = ("The quick brown fox jumps over the lazy dog. The dog barks and the fox runs away. "
        "Running foxes are quick and clever.")
words = [w.lower() for w in word_tokenize(text) if w.isalpha()]
print("Total words      :", len(words))
print("Unique words     :", len(set(words)))
print("Lexical diversity:", round(len(set(words))/len(words), 3))
print("Avg word length  :", round(sum(map(len, words))/len(words), 2))
print("Top 5 frequent   :", Counter(words).most_common(5))
print("Longest word     :", max(words, key=len))

# Word generation: rule based prefix/suffix attachment
prefixes = ["un", "re", "pre", "dis"]
suffixes = ["ing", "ed", "er", "ly", "ness"]
def generate(root):
    out = [root]
    out += [p + root for p in prefixes]
    out += [root + s for s in suffixes]
    return out
for root in ["play", "happy", "kind"]:
    print(f"\nRoot '{root}' ->", generate(root))

# Generation via WordNet lemmas (derivationally related forms)
print("\nDerivationally related forms (WordNet):")
```

```
for w in ["act", "create", "decide"]:
    forms = {l.name() for s in wn.synsets(w) for lem in s.lemmas()
             for l in lem.derivationally_related_forms()}
    print(f"  {w:7s}->", sorted(forms)[:10])
```

Result and Discussion

Output:

```
Total words      : 23
Unique words     : 16
Lexical diversity: 0.696
Avg word length  : 4.0
Top 5 frequent   : [('the', 4), ('quick', 2), ('fox', 2), ('dog', 2), ('and', 2)]
Longest word     : running

Root 'play' -> ['play', 'unplay', 'replay', 'preplay', 'display', 'playing', 'played',
'player', 'playly', 'playness']

Root 'happy' -> ['happy', 'unhappy', 'rehappy', 'prehappy', 'dishappy', 'happying', 'happyed',
'happyer', 'happyly', 'happyness']

Root 'kind' -> ['kind', 'unkind', 'rekind', 'prekind', 'diskind', 'kinding', 'kinded',
'kinder', 'kindly', 'kindness']

Derivationally related forms (WordNet):
  act    -> ['act', 'actable', 'acting', 'action', 'actor', 'actuate', 'move', 'play',
'playacting', 'playactor']
  create -> ['Creation', 'creation', 'creative', 'creator', 'creature', 'make', 'maker',
'making', 'producer', 'product']
  decide -> ['adjudicative', 'adjudicatory', 'deciding', 'decision', 'decisive',
'determinant', 'determination', 'judge', 'resolve', 'settlement']
```

Discussion: The sample text had 23 words of which 16 were unique (lexical diversity 0.696, average length 4.0). *the* was the most frequent word (4 times). Rule-based generation produced valid words such as *unhappy, replay, kindly, kindness, player*, but also invalid ones such as *playly, happyed, rehappy, kinded*, because plain concatenation ignores spelling rules and part-of-speech restrictions (*happy+ness* should be *happiness*). WordNet generation, in contrast, only returned real words (*act -> action, actor, acting*; *create -> creation, creative, creator*; *decide -> decision, decisive*). This shows that rule-based generation needs a validation step, e.g. a dictionary check.

Conclusion

Word statistics and generation were implemented; rule-based generation is flexible but produces invalid forms, whereas WordNet derivations are reliable.

Experiment No: 3

Experiment Name

Implementation of Word Sense Disambiguation (WSD) Using Python

Objective

To identify the correct meaning of a polysemous word in context using the Lesk algorithm in NLTK and WordNet.

Introduction

Many words have several senses (*bank* = financial institution or river side). Word Sense Disambiguation selects the sense that fits the context. The Lesk algorithm is a knowledge-based method: it picks the WordNet synset whose dictionary gloss (definition and examples) has the greatest word overlap with the words surrounding the target.

Algorithm

1. Take a sentence and a target word.
2. Tokenize the sentence to get the context words.
3. For every WordNet synset of the target, compute the number of words shared between its gloss and the context.
4. Return the synset with the highest overlap (`nltk.wsd.lesk`).
5. Print the sense name and its definition.

Code

```
from nltk.wsd import lesk
from nltk.tokenize import word_tokenize
from nltk.corpus import wordnet as wn

tests = [("I went to the bank to deposit my money", "bank"),
         ("The boat was tied up on the bank of the river", "bank"),
         ("He used a bat to hit the ball", "bat"),
         ("A bat flew out of the cave at night", "bat"),
         ("Python is a popular programming language", "python")]
for sent, w in tests:
    s = lesk(word_tokenize(sent), w)
    print(f"Sentence : {sent}\nWord     : {w}\nSense    : {s.name() if s else None}\nGloss
: {s.definition() if s else None}\n")
```

Result and Discussion

Output:

```
Sentence : I went to the bank to deposit my money
Word     : bank
Sense    : depository_financial_institution.n.01
Gloss    : a financial institution that accepts deposits and channels the money into lending
activities

Sentence : The boat was tied up on the bank of the river
Word     : bank
Sense    : bank.n.01
Gloss    : sloping land (especially the slope beside a body of water)
```

```
Sentence : He used a bat to hit the ball
Word     : bat
Sense    : bat.n.02
Gloss    : (baseball) a turn trying to get a hit

Sentence : A bat flew out of the cave at night
Word     : bat
Sense    : bat.v.03
Gloss    : have a turn at bat

Sentence : Python is a popular programming language
Word     : python
Sense    : python.n.02
Gloss    : a soothsaying spirit or a person who is possessed by such a spirit
```

Discussion: Two *bank* sentences were resolved correctly: the money sentence gave `depository_financial_institution.n.01` and the river sentence gave `bank.n.01` (sloping land beside water). However, *bat* in both sentences was resolved wrongly (`bat.n.02` "a turn at bat" and `bat.v.03`, instead of the club and the flying mammal), and *python* in a programming context was assigned a soothsaying-spirit sense, since WordNet has no programming-language sense for it. So 2 of 5 test cases were correct (40%). The results show the main limitation of Lesk: it depends on exact gloss overlap, so short contexts and glosses lacking relevant words give poor results. Better results need larger contexts, stop-word removal, embeddings or supervised models.

Conclusion

The Lesk algorithm works when gloss overlap is rich (*bank*) but fails on short contexts (*bat*, *python*), so it is only a baseline WSD method.

Experiment No: 4

Experiment Name

Implementation of Stemming Using the NLTK Toolkit

Objective

To reduce inflected words to their stems with the Porter, Lancaster and Snowball stemmers, and compare them with lemmatization.

Introduction

Stemming strips suffixes so that related word forms (*connect, connected, connecting*) map to one representation, which improves recall in information retrieval. Porter is moderate, Lancaster is the most aggressive, and Snowball (Porter2) is an improved Porter. Unlike lemmatization, stemming does not guarantee that the result is a real word.

Algorithm

1. Create the stemmer objects and a WordNet lemmatizer.
2. For each word in the list apply each stemmer.
3. Apply the lemmatizer with POS = verb.
4. Print the results as a comparison table.

Code

```
from nltk.stem import PorterStemmer, LancasterStemmer, SnowballStemmer, WordNetLemmatizer
words = ["running", "runs", "ran", "easily", "fairly", "studies", "studying", "happiness",
         "generously", "connection", "connected", "connecting", "better", "caring"]
p, l, s, lem = PorterStemmer(), LancasterStemmer(), SnowballStemmer("english"), WordNetLemmatizer()
print(f"{'Word':14s}{'Porter':12s}{'Lancaster':12s}{'Snowball':12s}{'Lemma(v)':12s}")
for w in words:
    print(f"{w:14s}{p.stem(w):12s}{l.stem(w):12s}{s.stem(w):12s}{lem.lemmatize(w,'v'):12s}")
```

Result and Discussion

Output:

```
Word          Porter      Lancaster   Snowball    Lemma(v)
running       run         run         run         run
runs          run         run         run         run
ran           ran         ran         ran         run
easily        easili      easy        easili      easily
fairly        fairli      fair        fair        fairly
studies       studi       study       studi       study
studying      studi       study       studi       study
happiness     happi       happy       happi       happiness
generously    gener       gen         generous    generously
connection    connect     connect     connect     connection
connected     connect     connect     connect     connect
connecting    connect     connect     connect     connect
better        better      bet         better      better
caring        care        car         care        care
```

Discussion: All stemmers reduced *running, runs* and *connected/connecting/connection* to the same stem (*run*, *connect*). Differences appeared elsewhere: Lancaster was the most aggressive (*generously -> gen*, *caring -> car*, *better -> bet*), producing over-stemming errors, whereas Snowball handled *generously -> generous* and *fairly -> fair* better than Porter (*gener*, *fairli*). Porter and Snowball produced non-words such as *studi, happi, easili*. The irregular verb *ran* was only handled by the lemmatizer (*run*). Hence stemmers are fast but crude, and lemmatization is preferable when valid words are needed.

Conclusion

Porter, Snowball and Lancaster stemmers reduce word variants quickly; Lancaster over-stems and lemmatization gives valid words.

Experiment No: 5

Experiment Name

Implementation of Part-of-Speech (POS) Tagging and Word POS Identification Using NLTK

Objective

To assign grammatical tags to the words of a sentence with NLTK's perceptron tagger and to look up the POS of individual words.

Introduction

POS tagging labels each token as noun, verb, adjective, etc. It supports parsing, named-entity recognition and information extraction. NLTK's default tagger is an averaged perceptron trained on the Penn Treebank, producing Penn tags (NN, VBZ, JJ) which can be mapped to the coarser Universal tagset (NOUN, VERB, ADJ).

Algorithm

1. Tokenize the sentence.
2. Call `nltk.pos_tag` to get (word, tag) pairs.
3. Re-run with `tagset='universal'`.
4. Convert to a dictionary and look up chosen words.
5. Count tags with `Counter` and print the tag definition.

Code

```
import nltk
from nltk.tokenize import word_tokenize
text = "The quick brown fox jumps over the lazy dog while she was reading a book."
tags = nltk.pos_tag(word_tokenize(text))
print("Tagged:", tags)
print("\nUniversal tagset:", nltk.pos_tag(word_tokenize(text), tagset="universal"))
print("\nWord -> POS lookup")
for w in ["fox", "jumps", "quick", "over", "she"]:
    print(f"  {w:6s}", dict(tags)[w])
from collections import Counter
print("\nTag counts:", Counter(t for _, t in tags))
print("\nTag meaning: NN noun, VBZ verb 3rd sg, JJ adjective, DT determiner, IN preposition,
PRP pronoun")
nltk.help.upenn_tagset("NN")
```

Result and Discussion

Output:

```
Tagged: [('The', 'DT'), ('quick', 'JJ'), ('brown', 'NN'), ('fox', 'NN'), ('jumps', 'VBZ'),
('over', 'IN'), ('the', 'DT'), ('lazy', 'JJ'), ('dog', 'NN'), ('while', 'IN'), ('she', 'PRP'),
('was', 'VBD'), ('reading', 'VBG'), ('a', 'DT'), ('book', 'NN'), ('.', '.')]

Universal tagset: [('The', 'DET'), ('quick', 'ADJ'), ('brown', 'NOUN'), ('fox', 'NOUN'),
('jumps', 'VERB'), ('over', 'ADP'), ('the', 'DET'), ('lazy', 'ADJ'), ('dog', 'NOUN'),
('while', 'ADP'), ('she', 'PRON'), ('was', 'VERB'), ('reading', 'VERB'), ('a', 'DET'),
('book', 'NOUN'), ('.', '.')]

Word -> POS lookup
  fox    NN
```

```
  jumps  VBZ
  quick  JJ
  over   IN
  she    PRP

Tag counts: Counter({'NN': 4, 'DT': 3, 'JJ': 2, 'IN': 2, 'VBZ': 1, 'PRP': 1, 'VBD': 1, 'VBG':
1, '.': 1})

Tag meaning: NN noun, VBZ verb 3rd sg, JJ adjective, DT determiner, IN preposition, PRP
pronoun
NN: noun, common, singular or mass
```

Discussion: The 16 tokens were tagged correctly for the most part: *The/DT quick/JJ fox/NN jumps/VBZ over/IN lazy/JJ dog/NN she/PRP was/VBD reading/VBG book/NN*. One questionable tag is *brown*, tagged NN instead of JJ in this context. Tag counts were NN=4, DT=3, JJ=2, IN=2. The Universal tagset collapsed VBZ/VBD/VBG into VERB and IN into ADP. The tagger therefore achieves good accuracy on ordinary text but can err on words whose category depends on subtle context.

Conclusion

NLTK's POS tagger tagged the sentence accurately and the Universal tagset gave a simpler view of the same information.

Experiment No: 6

Experiment Name

Implementation of Morphological Analysis, N-Gram Generation, and N-Gram Smoothing Using NLTK

Objective

To analyse word morphology, to generate bigrams and trigrams, and to apply Laplace (add-one) smoothing to n-gram probabilities.

Introduction

Morphological analysis breaks words into stems/lemmas and affixes. N-grams are contiguous sequences of *n* tokens used in language models to estimate P(w_n | w_(n-1)). Maximum-likelihood estimates give zero probability to unseen n-grams, which makes whole sentences impossible. Smoothing redistributes some probability mass to unseen events; Laplace smoothing adds one to every count: P = (C(w1,w2)+1)/(C(w1)+V).

Algorithm

1. Apply the stemmer and lemmatizer to sample words and compare.
2. Tokenize the corpus and generate bigrams and trigrams with `nltk.ngrams`.
3. Count unigrams and bigrams, set V = vocabulary size.
4. Compute MLE = C(w1,w2)/C(w1).
5. Compute Laplace = (C(w1,w2)+1)/(C(w1)+V).
6. Train an NLTK `Laplace` bigram model and compute probabilities and perplexity of a test sentence.

Code

```
import math
from collections import Counter
from nltk import ngrams, FreqDist
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer, PorterStemmer
from nltk.lm import Laplace, MLE
from nltk.lm.preprocessing import padded_everygram_pipeline

# Morphological analysis
lem, st = WordNetLemmatizer(), PorterStemmer()
print("Morphological analysis:")
for w in ["unhappiness", "replayed", "cars", "mice", "walking"]:
    print(f"  {w:12s} stem={st.stem(w):9s} lemma(n)={lem.lemmatize(w):9s} lemma(v)={lem.lemmat
ize(w,'v')}")

corpus = ["i love natural language processing", "i love machine learning",
          "natural language processing is fun", "machine learning is fun"]
tok = [word_tokenize(s) for s in corpus]
text = " ".join(corpus).split()
print("\nBigrams :", list(ngrams(text[:7], 2)))
print("Trigrams:", list(ngrams(text[:7], 3)))

uni = Counter(text); bi = Counter(ngrams(text, 2)); V = len(uni)
def p_mle(w1, w2): return bi[(w1, w2)] / uni[w1] if uni[w1] else 0
def p_add1(w1, w2): return (bi[(w1, w2)] + 1) / (uni[w1] + V)
print(f"\nVocabulary size V = {V}")
```

```
print(f"{'bigram':28s}{'count':>6s}{'MLE':>9s}{'Laplace':>10s}")
for pair in [("i","love"), ("love","natural"), ("machine","learning"), ("learning","processing
"), ("fun","i")]:
    print(f"{str(pair):28s}{bi[pair]:6d}{p_mle(*pair):9.3f}{p_add1(*pair):10.3f}")

# NLTK language model with Laplace smoothing; perplexity on an unseen sentence
train, vocab = padded_everygram_pipeline(2, tok)
lm = Laplace(2); lm.fit(train, vocab)
test = ["i", "love", "machine", "learning"]
print("\nNLTK Laplace P(love|i)       =", round(lm.score("love", ["i"]), 4))
print("NLTK Laplace P(processing|i) =", round(lm.score("processing", ["i"]), 4), "(unseen
bigram, non-zero)")
from nltk.util import bigrams
print("Perplexity of 'i love machine learning' =", round(lm.perplexity(list(bigrams(test))),
3))
```

Result and Discussion

Output:

```
Morphological analysis:
  unhappiness  stem=unhappi   lemma(n)=unhappiness lemma(v)=unhappiness
  replayed     stem=replay    lemma(n)=replayed  lemma(v)=replay
  cars         stem=car       lemma(n)=car       lemma(v)=cars
  mice         stem=mice      lemma(n)=mouse     lemma(v)=mice
  walking      stem=walk      lemma(n)=walking   lemma(v)=walk

Bigrams : [('i', 'love'), ('love', 'natural'), ('natural', 'language'), ('language',
'processing'), ('processing', 'i'), ('i', 'love')]
Trigrams: [('i', 'love', 'natural'), ('love', 'natural', 'language'), ('natural', 'language',
'processing'), ('language', 'processing', 'i'), ('processing', 'i', 'love')]

Vocabulary size V = 9
bigram                       count      MLE   Laplace
('i', 'love')                    2    1.000     0.273
('love', 'natural')              1    0.500     0.182
('machine', 'learning')          2    1.000     0.273
('learning', 'processing')       0    0.000     0.091
('fun', 'i')                     0    0.000     0.091

NLTK Laplace P(love|i)       = 0.2143
NLTK Laplace P(processing|i) = 0.0714 (unseen bigram, non-zero)
Perplexity of 'i love machine learning' = 5.342
```

Discussion: Stemming and lemmatization gave *unhappiness -> unhappi*, *replayed -> replay*, *mice -> mouse* (only by lemmatizer, needing the right POS). For the 4-sentence corpus (V = 9), MLE gave P(love|i) = 1.0 while unseen bigrams such as (learning, processing) had probability 0. After Laplace smoothing they received 0.091 and the seen bigram (i, love) fell from 1.0 to 0.273, showing how probability mass is moved from seen to unseen events. The NLTK model gave P(love|i) = 0.2143 (its counts include sentence padding symbols) and a perplexity of 5.342 for the test sentence. Add-one smoothing is simple but takes too much mass from seen events for large vocabularies; Good-Turing or Kneser-Ney are better.

Conclusion

N-gram probabilities were computed, and Laplace smoothing removed zero probabilities at the cost of lowering seen probabilities.

Experiment No: 7

Experiment Name

POS Tag Ambiguity Resolution Using Transition and Emission Probabilities in NLP

Objective

To resolve POS ambiguity of words like *fish*, *saw*, *can* by finding the most probable tag sequence with a Hidden Markov Model and the Viterbi algorithm.

Introduction

Many words are ambiguous (*saw* can be a noun or a verb). An HMM tagger models the tag sequence with transition probabilities P(tag_i | tag_(i-1)) and the word given a tag with emission probabilities P(word | tag). The Viterbi algorithm uses dynamic programming to find the tag sequence maximising the product of these probabilities in O(T x N^2) time.

Algorithm

1. Define states (N, V, D), start probabilities, transition matrix and emission matrix (toy values).
2. Initialise V[0][s] = start[s] x emit[s][w0].
3. For each next word t and state s: V[t][s] = max over r of V[t-1][r] x trans[r][s] x emit[s][w_t], storing the best previous state as a back-pointer.
4. Pick the state with the highest final score.
5. Follow the back-pointers to recover the tag path.

Code

```
# Viterbi decoding for POS tag ambiguity using transition and emission probabilities
states = ["N", "V", "D"]
start = {"N": 0.3, "V": 0.1, "D": 0.6}
trans = {"D": {"N": 0.8, "V": 0.1, "D": 0.1},
         "N": {"N": 0.2, "V": 0.6, "D": 0.2},
         "V": {"N": 0.4, "V": 0.1, "D": 0.5}}
emit = {"D": {"the": 0.7, "a": 0.3},
        "N": {"fish": 0.3, "can": 0.1, "saw": 0.1, "book": 0.2, "dog": 0.3},
        "V": {"fish": 0.2, "can": 0.3, "saw": 0.3, "book": 0.2}}

def viterbi(obs):
    V = [{s: start[s] * emit[s].get(obs[0], 1e-6) for s in states}]
    back = [{}]
    for t in range(1, len(obs)):
        V.append({}); back.append({})
        for s in states:
            p, prev = max((V[t-1][r] * trans[r][s] * emit[s].get(obs[t], 1e-6), r) for r in st
ates)
            V[t][s], back[t][s] = p, prev
    last = max(V[-1], key=V[-1].get)
    path = [last]
    for t in range(len(obs)-1, 0, -1): path.insert(0, back[t][path[0]])
    return path, V

for sent in ["the dog saw a fish", "the fish can", "the book fish"]:
    obs = sent.split()
    path, V = viterbi(obs)
    print(f"\nSentence: {sent}")
    for t, w in enumerate(obs):
```

```
        print(f"  {w:5s} " + "  ".join(f"{s}:{V[t][s]:.6f}" for s in states))
    print("  Best tags:", list(zip(obs, path)))
```

Result and Discussion

Output:

```
Sentence: the dog saw a fish
  the   N:0.000000  V:0.000000  D:0.420000
  dog   N:0.100800  V:0.000000  D:0.000000
  saw   N:0.002016  V:0.018144  D:0.000000
  a     N:0.000000  V:0.000000  D:0.002722
  fish  N:0.000653  V:0.000054  D:0.000000
  Best tags: [('the', 'D'), ('dog', 'N'), ('saw', 'V'), ('a', 'D'), ('fish', 'N')]

Sentence: the fish can
  the   N:0.000000  V:0.000000  D:0.420000
  fish  N:0.100800  V:0.008400  D:0.000000
  can   N:0.002016  V:0.018144  D:0.000000
  Best tags: [('the', 'D'), ('fish', 'N'), ('can', 'V')]

Sentence: the book fish
  the   N:0.000000  V:0.000000  D:0.420000
  book  N:0.067200  V:0.008400  D:0.000000
  fish  N:0.004032  V:0.008064  D:0.000000
  Best tags: [('the', 'D'), ('book', 'N'), ('fish', 'V')]
```

Discussion: The ambiguous words were resolved using context: in *the dog saw a fish*, *saw* became a verb (V: 0.018 vs N: 0.002) because a noun is typically followed by a verb, and *fish* became a noun after the determiner. In *the fish can*, *fish* was a noun and *can* a verb. In *the book fish*, *book* was N and *fish* V (0.008064 vs 0.004032). The same word received different tags according to neighbours, which demonstrates that transition probabilities supply the context that emission probabilities alone lack. The probabilities here are hand-set toy values; real taggers estimate them from a tagged corpus, and unseen words need smoothing (a 1e-6 floor was used).

Conclusion

The Viterbi algorithm resolved ambiguous tags by combining emission and transition probabilities.

Experiment No: 8

Experiment Name

Implementation of Grapheme-to-Phoneme (G2P) Conversion

Objective

To convert written words into phoneme sequences using the CMU Pronouncing Dictionary with a rule-based fallback for out-of-vocabulary words.

Introduction

G2P conversion maps spelling (graphemes) to pronunciation (phonemes), a key component of speech synthesis and recognition. English spelling is irregular, so systems combine a pronunciation dictionary (CMUdict, ARPAbet symbols with stress digits) with letter-to-sound rules or neural models for unknown words.

Algorithm

1. Load CMUdict through NLTK.
2. For each word, lower-case it and look it up; if found return the first pronunciation.
3. Otherwise scan the word left to right, matching the longest grapheme in an ordered rule table (e.g. *sh, ch, th, ee*) and emit its phoneme.
4. Print the phoneme sequence and its source.

Code

```
# Grapheme-to-Phoneme conversion: dictionary lookup (CMUdict) + rule-based fallback for OOV
words
import re
from nltk.corpus import cmudict
cmu = cmudict.dict()

# Simplified letter-to-sound rules (ordered; longest grapheme first)
RULES = [("tch","CH"),("sh","SH"),("ch","CH"),("th","TH"),("ph","F"),("ng","NG"),("ck","K"),("
qu","K W"),
         ("ee","IY"),("oo","UW"),("ai","EY"),("ay","EY"),("oa","OW"),("ou","AW"),
         ("a","AE"),("e","EH"),("i","IH"),("o","AA"),("u","AH"),("b","B"),("c","K"),("d","D"),
("f","F"),
         ("g","G"),("h","HH"),("j","JH"),("k","K"),("l","L"),("m","M"),("n","N"),("p","P"),("r
","R"),
         ("s","S"),("t","T"),("v","V"),("w","W"),("x","K S"),("y","Y"),("z","Z")]

def rule_g2p(word):
    out, i = [], 0
    while i < len(word):
        for g, p in RULES:
            if word.startswith(g, i):
                out.append(p); i += len(g); break
        else:
            i += 1
    return " ".join(out).split()

def g2p(word):
    w = word.lower()
    if w in cmu: return cmu[w][0], "dictionary"
    return rule_g2p(w), "rules"
```

```
for sent in ["speech technology changes language", "the quick brown fox", "blorpting chatbots
tokenizer"]:
    print("\nSentence:", sent)
    for w in re.findall(r"[a-z']+", sent):
        ph, src = g2p(w)
        print(f"  {w:12s} /{' '.join(ph):32s}/ [{src}]")
```

Result and Discussion

Output:

```
Sentence: speech technology changes language
  speech       /S P IY1 CH                      / [dictionary]
  technology   /T EH0 K N AA1 L AH0 JH IY0      / [dictionary]
  changes      /CH EY1 N JH AH0 Z               / [dictionary]
  language     /L AE1 NG G W AH0 JH             / [dictionary]

Sentence: the quick brown fox
  the          /DH AH0                          / [dictionary]
  quick        /K W IH1 K                       / [dictionary]
  brown        /B R AW1 N                       / [dictionary]
  fox          /F AA1 K S                       / [dictionary]

Sentence: blorpting chatbots tokenizer
  blorpting    /B L AA R P T IH NG              / [rules]
  chatbots     /CH AE T B AA T S                / [rules]
  tokenizer    /T AA K EH N IH Z EH R           / [rules]
```

Discussion: Dictionary words were converted accurately, e.g. *speech -> S P IY1 CH*, *language -> L AE1 NG G W AH0 JH*, with stress markers. The out-of-vocabulary words *blorpting, chatbots, tokenizer* were handled by rules: *chatbots -> CH AE T B AA T S* is acceptable, but *tokenizer -> T AA K EH N IH Z EH R* is wrong (should be roughly T OW K AH N AY Z ER) because the simple rules ignore vowel length, the silent *e* and stress. Dictionary lookup is therefore accurate but limited in coverage, and rules are general but inaccurate; neural G2P models (e.g. g2p-en) combine both advantages.

Conclusion

G2P using CMUdict is accurate for known words; the rule fallback is only approximate for unknown words.

Experiment No: 9

Experiment Name

Implementation of Text-to-Speech (TTS) for Converting Text into Audio

Objective

To convert written text into a spoken audio file using the gTTS library.

Introduction

Text-to-Speech synthesis generates speech from text. A typical pipeline performs text normalisation, G2P conversion, prosody prediction and waveform generation. The gTTS library calls Google's online TTS service and returns an MP3 file, so the pipeline details are hidden behind a simple API.

Algorithm

1. Define the input text and language code.
2. Create a `gTTS` object with the text.
3. Save the audio as MP3.
4. Convert the MP3 to 16 kHz mono WAV with ffmpeg (the format needed by the ASR experiment).
5. Report file sizes and duration.

Code

```
# Text-to-Speech using gTTS; output saved as MP3 and converted to 16 kHz WAV for experiment 10
import subprocess, os
from gtts import gTTS
text = "Natural language processing allows computers to understand human speech."
os.makedirs("../audio", exist_ok=True)
gTTS(text=text, lang="en").save("../audio/tts_output.mp3")
subprocess.run(["ffmpeg","-y","-loglevel","error","-i","../audio/tts_output.mp3","-
ar","16000","-ac","1","../audio/tts_output.wav"], check=True)
print("Input text :", text)
for f in ["tts_output.mp3","tts_output.wav"]:
    print(f"Created ../audio/{f}: {os.path.getsize('../audio/'+f)} bytes")
print(subprocess.run(["ffprobe","-v","error","-show_entries","format=duration","-of","default=
nw=1","../audio/tts_output.wav"],capture_output=True,text=True).stdout.strip())
```

Result and Discussion

Output:

```
Input text : Natural language processing allows computers to understand human speech.
Created ../audio/tts_output.mp3: 44736 bytes
Created ../audio/tts_output.wav: 179022 bytes
duration=5.592000
```

Discussion: The sentence "Natural language processing allows computers to understand human speech." was synthesised into `audio/tts_output.mp3` (44,736 bytes) and converted to a 16 kHz mono WAV of 179,022 bytes with a duration of 5.59 s. The audio was verified by feeding it to the recogniser in Experiment 10, which transcribed it perfectly. gTTS needs an Internet connection

and only the sample sentence was sent to the service; offline engines such as pyttsx3/eSpeak or Piper are alternatives when privacy matters. Voice, speed and emotion control are limited.

Conclusion

Text was converted to a clear audio file of 5.59 s using gTTS.

Experiment No: 10

Experiment Name

Implementation of Automatic Speech Recognition (ASR) for Converting Audio into Text

Objective

To convert a speech audio file into text using the offline Vosk (Kaldi) recogniser and evaluate it with word error rate (WER).

Introduction

ASR maps an acoustic signal to a word sequence. Modern systems combine an acoustic model, a pronunciation lexicon and a language model. Vosk provides small offline Kaldi models. Quality is measured by WER = (substitutions + deletions + insertions) / number of reference words, computed with the edit-distance algorithm.

Algorithm

1. Load the Vosk English model.
2. Open the 16 kHz mono WAV file produced in Experiment 9.
3. Feed the audio in chunks of 4000 frames to the recogniser.
4. Read the final JSON result: text and per-word timestamps/confidence.
5. Compute WER against the reference with dynamic-programming edit distance.

Code

```
# Offline ASR with Vosk (Kaldi). Transcribes audio and reports word error rate against the
reference.
import wave, json, sys
from vosk import Model, KaldiRecognizer, SetLogLevel
SetLogLevel(-1)
MODEL = sys.argv[1]
reference = "natural language processing allows computers to understand human speech"
wf = wave.open("../audio/tts_output.wav", "rb")
rec = KaldiRecognizer(Model(MODEL), wf.getframerate()); rec.SetWords(True)
while True:
    data = wf.readframes(4000)
    if not data: break
    rec.AcceptWaveform(data)
res = json.loads(rec.FinalResult())
hyp = res["text"]
print("Reference :", reference)
print("Recognised:", hyp)
for w in res.get("result", []):
    print(f"  {w['word']:12s} {w['start']:.2f}-{w['end']:.2f}s  conf={w['conf']:.2f}")

def wer(r, h):
    r, h = r.split(), h.split()
    d = [[i+j if i*j == 0 else 0 for j in range(len(h)+1)] for i in range(len(r)+1)]
    for i in range(1, len(r)+1):
        for j in range(1, len(h)+1):
            d[i][j] = min(d[i-1][j]+1, d[i][j-1]+1, d[i-1][j-1]+(r[i-1] != h[j-1]))
    return d[-1][-1] / len(r)
print(f"Word Error Rate: {wer(reference, hyp):.2%}")
```

Result and Discussion

Output:

```
Reference : natural language processing allows computers to understand human speech
Recognised: natural language processing allows computers to understand human speech
  natural      0.27-0.87s  conf=1.00
  language     0.87-1.44s  conf=1.00
  processing   1.44-2.28s  conf=1.00
  allows       2.28-2.70s  conf=1.00
  computers    2.70-3.42s  conf=1.00
  to           3.42-3.60s  conf=1.00
  understand   3.60-4.23s  conf=1.00
  human        4.23-4.65s  conf=1.00
  speech       4.65-5.31s  conf=1.00
Word Error Rate: 0.00%
```

Discussion: The recogniser output was identical to the reference: "natural language processing allows computers to understand human speech". Every word had confidence 1.00 and timestamps across 0.27 s to 5.31 s, so WER = 0.00%. The result is favourable because the input is clean synthetic speech with one speaker and no noise; real recordings with noise, accents or overlapping speakers would give a higher WER. The text is lower-case without punctuation, so a punctuation/casing post-processing step is needed for presentable transcripts.

Conclusion

Vosk transcribed the audio with 0% WER on clean speech, completing the TTS-to-ASR round trip.